FADENA
FACULTAD DE DEFENSA NACIONAL

SQL Básico

Temario

- Introducción. Tipos de Sentencias.
- Sentencias de Definición de Datos (DDL): CREATE, ALTER, DROP.
    - Creación de bases de datos y esquemas.
    - Creación de tablas.
    - Modificación y borrado de tablas.
- Sentencias de Manipulación de Datos (DML): INSERT, UPDATE, DELETE, SELECT.
    - Inserción de datos.
    - Actualización de datos.
    - Eliminación de registros (filas).
    - Consulta de datos.

Introducción

- **SQL** (Structured Query Language) es un lenguaje estándar de definición, manipulación y control de bases de datos relacionales.
- Se basa en operaciones del álgebra y el cálculo relacional -con operadores adicionales-: así define consultas para recuperar o modificar información de bases de datos. Además, es posible definir la estructura de los datos, modificarla, y especificar restricciones.
- SGBD que usan SQL: Oracle, Microsoft SQL Server, DB2 de IBM, MySQL, MariaDB, PostgreSQL, SQLite, entre otros.
- Las distintas implementaciones de SQL pueden diferenciarse en detalles, o pueden admitir sólo un subconjunto del lenguaje completo.

Introducción

- **Tipos de sentencias:**
  - **DDL** (Data Definition Language): definir el esquema de bases de datos, crear tablas (con sus campos), alterar su definición y eliminarlas.
  - **DML** (Data Maniplation Language): insertar datos en las tablas, consultarlos, modificarlos y borrarlos.
  - **DCL** (Data Control Language): conceder o revocar privilegios de acceso a usuarios.

SENTENCIAS DE DEFINICIÓN DE DATOS (DDL)

- Permiten crear y borrar una base (o esquema); y crear, modificar y borrar las diferentes tablas que la componen. Palabras clave: **CREATE, ALTER, DROP**.

CREACIÓN Y BORRADO DE BASES DE DATOS Y ESQUEMAS

- El estándar SQL no dispone la sentencia de base de datos como tal (lo hace para esquemas). Sin embargo, la mayoría de SGBD incorporan el concepto de base de datos.
    - **Crear**: **CREATE SCHEMA** nombre_esquema; / **CREATE DATABASE** nombre_base;
    - **Borrar**: **DROP SCHEMA** nombre_esquema; / **DROP DATABASE** nombre_base;

SENTENCIAS DE DEFINICIÓN DE DATOS (DDL)

CREACIÓN DE TABLAS

- Se utiliza la sentencia **CREATE TABLE**:

CREATE TABLE nombre_tabla

(definición_columna

[, definición_columna...]

[, **restricciones_tabla**]);

nombre_columna tipo_datos [DEFAULT expr] [restricciones_columna]

Permite asignar un **valor por defecto**, que se utilizará cuando en una inserción no se especifique valor para esa columna: **expr** (puede ser NULL si el campo lo permite).

CHAR(n): Cadenas longitud fija hasta n caracteres.
VARCHAR(n): Cadenas longitud variable hasta n caracteres.
INTEGER, BIGINT, SMALLINT: Números enteros.
NUMERIC(n, m), DECIMAL(n, m), REAL, FLOAT: Números con coma.
DATE, TIME, DATETIME: Fechas, horas o ambas.
BIT, BLOB, y muchos más (varían según el SGBD).

(*) lo que esté entre corchetes [...] son elementos opcionales

(*) Algunos SGBD permiten indicar si un campo entero es autoincremental.

SENTENCIAS DE DEFINICIÓN DE DATOS (DDL)

CREACIÓN DE TABLAS (Restricciones)

Restricciones de Columna – [restricciones_columna] **:**

- **NOT NULL:** La columna **no** puede tener **valores nulos**.
- **NULL:** La columna **admite nulos** (se toma por defecto si no se especifica NOT NULL).
- **UNIQUE:** La columna **no** puede tener **valores repetidos**. Es una clave alternativa.
- **PRIMARY KEY:** La columna **no** puede tener **valores repetidos ni nulos**. Es la clave primaria.
- **REFERENCES tabla [(columna)]:** La columna es la **clave foránea** de la columna de la tabla especificada.
- **CHECK (condiciones):** La columna debe cumplir las condiciones especificadas.

SENTENCIAS DE DEFINICIÓN DE DATOS (DDL)

CREACIÓN DE TABLAS (Restricciones)

Restricciones de Tabla - [restricciones_tabla] **:**

- **UNIQUE (columna [, columna...]):** El conjunto de las columnas especificadas no puede tener valores repetidos. Es una clave alternativa.
- **PRIMARY KEY (columna [, columna...]):** El conjunto de las columnas especificadas no puede tener valores nulos ni repetidos. Es una clave primaria.
- **FOREIGN KEY (columna [, columna...]) REFERENCES tabla [(columna2 [, columna2...])]:** El conjunto de las columnas especificadas es una clave foránea que referencia la clave primaria formada por el conjunto de las columnas2 de la tabla dada.
- **CHECK (condiciones):** La tabla debe cumplir las condiciones especificadas.

(*) Si bien las restricciones de clave (primaria, foránea) como restricciones de tabla sirven para claves compuestas, suelen usarse incluso para claves simples.

SENTENCIAS DE DEFINICIÓN DE DATOS (DDL)

CREACIÓN DE TABLAS (Restricciones)

- **Nombre de restricción:** Se puede anteponer **CONSTRAINT nombre_restricción**, para nombrar la restricción. Si se omite, el sistema generará un nombre automáticamente.
- **Claves foráneas:** Si se definió una clave foránea, cuando se agrega o actualiza un valor en esta columna, se comprueba que dicho valor existe en la tabla referenciada. Además, **se pueden establecer reglas de comportamiento** para los casos en que se borra o actualiza el valor referenciado. Sintaxis completa para esto:

FOREIGN KEY clave_secundaria **REFERENCES** tabla [(clave_primaria)]

[ON DELETE {NO ACTION | CASCADE | SET DEFAULT | SET NULL}]

[ON UPDATE {NO ACTION | CASCADE | SET DEFAULT | SET NULL}]

NO ACTION (o RESTRICT): corresponde a la política de restricción (produce un error por intento de violación de restricción).

CASCADE: borra/actualiza las filas que hacen referencia al valor borrado/actualizado.

SET NULL: pone un nulo en las filas donde se hacía referencia al valor borrado/actualizado.

SET DEFAULT: pone el valor por defecto en las filas donde se hacía referencia al valor borrado/actualizado.

SENTENCIAS DE DEFINICIÓN DE DATOS (DDL)

CREACIÓN DE TABLAS (Ejemplo):

```
CREATE TABLE Productos (
        codigo_producto INTEGER,
        nombre_producto VARCHAR(50) UNIQUE NOT NULL,
        tipo VARCHAR(20),
        descripcion VARCHAR(255),
        precio REAL DEFAULT 10.0,
        fabrica INTEGER DEFAULT NULL,
        PRIMARY KEY (codigo_producto),
        FOREIGN KEY (fabrica) REFERENCES Fabricas(id_fabrica)
                ON DELETE RESTRICT ON UPDATE CASCADE,
        CONSTRAINT precio CHECK (precio>5.0)
);
```

(*) No siempre será necesario utilizar todas las posibilidades de definición, muchas veces alcanzará con los elementos básicos.

(*) Ver que, aunque la clave primaria no sea compuesta, se puede definir como restricción de tabla. Al definir como PRIMARY KEY, el sistema ya la asumirá como UNIQUE y NOT NULL.

SENTENCIAS DE DEFINICIÓN DE DATOS (DDL)

MODIFICACIÓN DE TABLAS

- Se utiliza la sentencia **ALTER TABLE** para agregar o eliminar una columna particular, establecer una nueva restricción, etc.:

ALTER TABLE nombre_tabla **ADD** nombre_columna definición_columna;
ALTER TABLE nombre_tabla **DROP COLUMN** nombre_columna;
ALTER TABLE nombre_tabla **ALTER COLUMN** nombre_columna definición_columna;

Agregar (ADD), borrar (DROP) o modificar (ALTER o MODIFY) una columna;

ALTER TABLE nombre_tabla **ADD CONSTRAINT** restricción_tabla;
ALTER TABLE nombre_tabla **DROP CONSTRAINT** nombre_restricción;

Agregar (ADD) o eliminar (DROP) restricciones de tabla.

(*) La sintaxis específica y las posibilidades dependerán de motor de base de datos usado.

SENTENCIAS DE DEFINICIÓN DE DATOS (DDL)

MODIFICACIÓN DE TABLAS (Ejemplos):

ALTER TABLE productos **ADD** marca VARCHAR(50);

ALTER TABLE productos **DROP COLUMN** fabrica;

ALTER TABLE productos **ADD CONSTRAINT** FK_Tipo **FOREIGN KEY** (tipo) **REFERENCES** tipos(id_tipo);

BORRADO DE TABLAS

- Se utiliza la sentencia **DROP TABLE** para borrar una tabla. Tener en cuenta que **elimina la tabla**, no sólo su contenido:

DROP TABLE nombre_tabla;

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

- Una vez creada la base de datos con sus tablas, se puede insertar, modificar y borrar los valores de las filas de las tablas. SQL ofrece las siguientes sentencias para esto:
  - **INSERT**: para insertar registros (filas) en la base de datos.
  - **UPDATE**: para modificar los valores de los campos indicados, en los registros que cumplan cierta condición.
  - **DELETE**: para eliminar los registros (filas) de una tabla que cumplan una condición.
- Una vez que se han insertado valores en las tablas, estos pueden ser consultados con:
  - **SELECT**: permite consultar registros de la base de datos que satisfagan una condición determinada.

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

INSERCIÓN DE DATOS

- Se pueden **insertar valores** en una tabla de grado n, mediante la sentencia **INSERT INTO**, de dos formas:

1 - Sin especificar los nombres de las columnas, sólo los valores:

```
INSERT INTO nombre_tabla
VALUES (v1, v2, ..., vn);
```

Donde los valores v1, v2, ..., vn se deben corresponder exactamente (en el mismo orden) con las columnas de la tabla donde se están insertando.

2 - Especificando el nombre de los $k \leq n$ atributos para los que insertamos valores:

```
INSERT INTO nombre_tabla (col1, col2, ..., colk)
VALUES (v1, v2, ..., vk);
```

En este caso, los valores no indicados explícitamente toman el valor NULL, o bien el valor por defecto si se definió mediante DEFAULT.

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

INSERCIÓN DE DATOS (Ejemplos):

INSERT INTO Productos

VALUES (1001, 'MatrixQ', ‘teclado’, ‘Teclado retroiluminado inalámbrico’, 6.75, NULL);

INSERT INTO Productos

VALUES (1002, 'SuperM', ‘monitor’, ‘Monitor LCD 24 Pulgadas’, 12.15, NULL);

O bien,

INSERT INTO Productos (codigo_producto, nombre_producto, tipo, precio)

VALUES (1003, 'Zitrix', ‘mouse’, 6.75);

(*) Cuando se insertan datos deben respetarse las restricciones establecidas (de entidad, referencial, dominio, etc.).

(*) Algunos SGBD permiten insertar varias filas en una misma tabla mediante una sola sentencia INSERT, con listas de valores separadas por coma.

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

ACTUALIZACIÓN DE DATOS

- Se pueden **modificar valores** en una tabla mediante la sentencia **UPDATE SET WHERE**:

```
UPDATE Nombre_tabla
    SET col1 = v1, col2 = v2, ..., coln = vn
WHERE condiciones;
```

Esto actualizará a los valores v1, ..., vn, las columnas col1, ..., coln, respectivamente, para todos los registros que cumplan las condiciones dadas.

Ejemplo:

```
UPDATE Productos
    SET precio = precio * 1.10
WHERE tipo = 'monitor';
```

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

ELIMINACIÓN DE REGISTROS

- Se pueden **eliminar registros (filas)** de una tabla mediante la sentencia **DELETE FROM WHERE**:

DELETE FROM Nombre_tabla

WHERE condiciones;

Esto eliminará los registros de la tabla que cumplan las condiciones dadas.

Ejemplo:

```
DELETE FROM Productos
WHERE codigo_producto = 1005;
```

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

Sobre las actualizaciones y borrados:

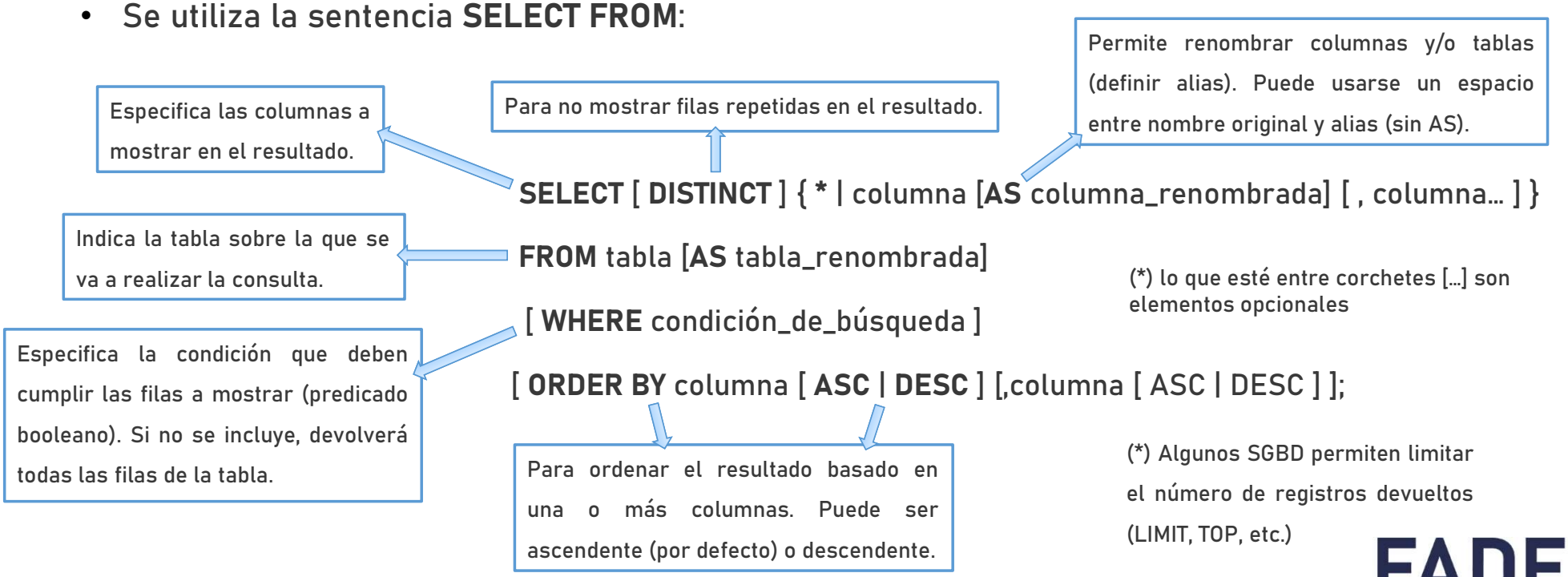
- Si se quisiera actualizar campos para todas las filas de una tabla; o borrar todas las filas de una tabla, se usaría sólo las sentencias UPDATE o DELETE FROM, **sin el WHERE**. **Si no es la intención, hay que prestar especial atención en evitarlo**, porque podría llevar a la pérdida de datos o modificaciones erróneas en toda la tabla.

- Para comprender completamente el funcionamiento de UPDATE y DELETE es necesario conocer bien el funcionamiento de la sentencia SELECT. Esto es así porque **para poder actualizar o eliminar datos que se han almacenado es preciso encontrarlos antes**. Por lo tanto, para estas sentencias se establecen las condiciones de búsqueda de dichos datos (**WHERE**) del mismo modo que las condiciones de búsqueda cuando se hace una consulta.

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS

- Se utiliza la sentencia **SELECT FROM**:



(*) lo que esté entre corchetes [...] son elementos opcionales

(*) Algunos SGBD permiten limitar el número de registros devueltos (LIMIT, TOP, etc.)

FADENA
FACULTAD DE DEFENSA NACIONAL

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS (Operadores para la cláusula WHERE)

Para definir las condiciones (que deben cumplir los campos) de la cláusula WHERE, se pueden usar distintos tipos de operadores.

Operadores lógicos: permiten formar **condiciones más complejas a partir de otras**. Así, dadas tres condiciones a, b y c:

- **a AND b**: conjunción, evalúa las condiciones a y b y devuelve verdad si ambas son ciertas.
- **a OR b**: disyunción lógica, evalúa a y b y devuelve verdad si alguna de las dos es cierta.
- **NOT c**: negación lógica, devuelve el valor lógico contrario a la evaluación de c.

(*) El orden de precedencia de los operadores lógicos es NOT, luego AND, y finalmente OR. Si se necesita otro orden, se tiene que usar paréntesis.

Operadores de comparación: para cada condición simple:

- **=** Igual que, **<>** Distinto de
- **<** Menor que, **>** Mayor que
- **<=** Menor o igual que, **>=** Mayor o igual que

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS (Operadores para la cláusula WHERE)

Otros operadores especiales: también se pueden emplear operadores especiales como:

- **BETWEEN: columna BETWEEN a AND b**, devuelve los registros en los que el valor de la columna esté entre los límites a y b, ambos inclusive. Por ej.: precio BETWEEN 5.0 AND 20.0.
- **IN: columna IN (valor1, …, valorN)**, devuelve los registros donde la columna indicada coincide con alguno de los dados en una lista. Por ej.: provincia IN ('Mendoza', 'Tucumán', 'Jujuy').
- **LIKE: columna LIKE característica**, permite comprobar si una columna de texto cumple ciertos patrones. En general la característica es un texto donde se usan "comodines": _ (guión bajo) para considerar caracteres individuales, y % para una secuencia de cero o más caracteres. Por ej., Apellido LIKE 'S%' (apellidos que empiezan con S).
- **IS NULL: columna IS NULL**, permite comprobar si un valor es nulo. Para saber si una columna no es nula, el operador es **IS NOT NULL**. Recordemos que un nulo no es un valor, implica la ausencia de valor.

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS (Operadores y Funciones para las cláusulas SELECT y WHERE)

En las cláusulas SELECT y WHERE, **además de columnas, se pueden incluir expresiones que contengan columnas y constantes**, así como funciones (dependerán del SGBD)

Operadores y funciones matemáticas:

- **Operadores entre números:** n + m (suma), n - m (resta), n * m (multiplicación), n / m (división), n % m (módulo o resto).
- **Funciones matemáticas:** ABS(n) (valor absoluto), SQRT(n) (raíz cuadrada), POWER(n, e) (potencia de n elevado a e), etc.

Operadores y funciones de fecha:

- **EXTRACT(parte FROM fecha) ó DATEPART(parte, fecha) ó DATE_PART(parte, fecha)**: extrae una parte de la fecha (año, mes, día).
- **YEAR(fecha), MONTH(fecha), DAY(fecha)**: extrae año, mes o día de la fecha.
- **DATEDIFF(fecha1, fecha2)**: calcula la diferencia entre dos fechas.
- **DATEADD(intervalo, n, f)**: suma o resta un intervalo especificado (días, meses, años) a una fecha.
- **CURRENT_DATE ó GETDATE()**: devuelve la fecha actual del sistema.
- Otros…

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS (Operadores y Funciones para las cláusulas SELECT y WHERE)

Operadores y funciones de cadenas de caracteres:

- **t || r ó t + r ó CONCAT(t, r,..)**: concatena cadenas.
- **UPPER(t) y LOWER(t)**: convierte a mayúsculas o minúsculas.
- **LEN(t) ó LENGTH(t)**: devuelve el número de caracteres.
- **REPLACE(cadena, a_buscar, a_reemplazar)**: reemplaza las ocurrencias de un texto por otro.
- **SUBSTRING(cadena, inicio, longitud)**: devuelve la subcadena especificada a partir de una posición.
- **LTRIM(t), RTRIM(t), TRIM(t)**: elimina espacios en blanco a la izquierda, derecha, o ambos.
- Otros…

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS (Ejemplos):

```
SELECT *
  FROM empelados;
```

```
SELECT nombre_empleado
   FROM empelados
WHERE categoría = 'primera'
    AND sueldo > 50.000;
```

```
SELECT *
  FROM empelados
ORDER BY nombre_empleado
LIMIT 10;
```

```
SELECT DISTINCT codcli, EXTRACT(month FROM fecha) AS meses
  FROM facturas
WHERE codcli IN (45, 54, 87, 102)
 ORDER BY fecha DESC;
```

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS – Funciones de Agregación:

SQL ofrece una serie de funciones que permiten contar datos o hacer cálculos sobre ellos. Se usan dentro de una cláusula SELECT y actúan sobre los valores de las columnas. Los más habituales son:

- **COUNT(*)**: Cuenta filas.
- **COUNT(columna)**: Cuenta valores no nulos de la columna.
- **COUNT(DISTINCT columna)**: Cuenta valores distintos y no nulos de la columna.
- **SUM(columna)**: Suma los valores de la columna.
- **MAX(columna):** Obtiene el valor máximo de la columna.
- **MIN(columna):** Obtiene el valor mínimo de la columna.
- **AVG(columna):** Obtiene la media de los valores de la columna.

Ejemplo:

```
SELECT MAX(precio) AS pr_máx_monitores,
       AVG(precio) AS pr_prom_monitores
  FROM Productos
WHERE tipo = 'monitor';
```

(*) Si no se especifica una restricción en la cláusula WHERE, las funciones de agregación se aplican sobre todas las filas de la tabla.

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS – Agrupación de Filas (Agrupamiento):

- La cláusula **GROUP BY** forma grupos con las **filas que tienen en común los valores de una o varias columnas**.
- Se mostrará una única fila por cada grupo.
- Se pueden aplicar funciones de agregación (SUM, MAX, MIN, AVG, COUNT) -> funciones de grupo. Se utilizan en la cláusula SELECT. Tendrán un valor para cada grupo.

- La cláusula **HAVING** permite filtrar los registros una vez agrupados, según una condición.
- Usa las funciones de grupo (cálculos agregados) para las condiciones de filtro.
- Similar al WHERE (que se puede aplicar antes para excluir aquellas filas que no se desea agrupar).

```
SELECT columna [ , columna ]
   FROM tabla
[ WHERE condición_de_búsqueda ]
[ GROUP BY columna [, columna ]
[ HAVING condición_para_el_grupo ] ]
[ ORDER BY columna [ ASC | DESC ] [,columna [ ASC | DESC ] ];
```

(*) Tanto en la cláusula SELECT como en HAVING sólo pueden aparecer las columnas que se incluyen en la cláusula GROUP BY, y/o las funciones de agregación.

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS – Agrupación de Filas (Ejemplo):

```
SELECT tipo, AVG(precio) AS precio_promedio
  FROM Productos
GROUP BY tipo
HAVING COUNT(*) >= 2;
```

Devolverá una fila por cada tipo de producto, mostrando el tipo y el precio promedio de los productos de ese tipo, siempre que haya al menos 2 productos de ese tipo. Si se omite el HAVING, se obtendrá una fila por cada tipo existente.

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS – Consultas a más de una tabla

En su forma más elemental se separan las **tablas involucradas por una coma (,)** :

```
SELECT columnas
  FROM tabla1, tabla2;
```

Es el producto cartesiano: Si tabla1 tiene n filas y r columnas; y tabla2 tiene m filas y s columnas, el resultado tendrá n × m filas y r + s columnas.

En general, **suele filtrarse por ciertas condiciones para restringir el resultado**. Por ej.:

```
SELECT *
  FROM facturas, clientes
WHERE facturas.codcli = clientes.codcli;
```

(*) Si bien esta forma (sintaxis) a quedado obsoleta en el estándar de SQL, puede encontrarse habitualmente su uso.

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS – Consultas a más de una tabla

Forma **más explícita y clara**, usando la cláusula **JOIN** (ó **INNER JON)**:

SELECT [DISTINCT] { * | [lista_columnas] }

FROM tabla1 **JOIN** tabla2 **ON** condiciones_vínculos_tablas

[WHERE condiciones_de_búsqueda]

- Permite identificar más claramente qué son restricciones (WHERE) y qué son condiciones de concatenación (JOIN y ON).
- La opción ON se usa para definir el criterio de selección: expresión booleana que retorna verdadero o falso (como en WHERE).

Ejemplo:

```
SELECT P.codigo_proyecto, P.precio, C.apellido
  FROM clientes C JOIN proyectos P
       ON C.codigo_cli = P.codigo_cliente
WHERE C.codigo_cli = 20;
```

(*) Devolverá el código del proyecto, su precio y apellido del cliente al que corresponde, donde el código del cliente sea 20.

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS – Consultas a más de una tabla

Se puede hacer **JOIN a la propia tabla** usando alias diferentes para la misma tabla:

```
SELECT A.nombre AS supervisor, B.nombre AS supervisado
  FROM Empleados A
        INNER JOIN Empleados B ON A.id = B.id_supevisor;
```

Se pueden usar **operadores que no sean el de igualdad**:

```
SELECT E1.codigo_empl, E1.apellido_empl
  FROM empleados E1 JOIN empleados E2 ON E1.sueldo > E2.sueldo
WHERE E2.codigo_empl = 5;
```

Si se quiere combinar **tres o más tablas**, hay que **ir haciendo combinaciones de tablas por pares**:

```
SELECT P.id_pedido, C.id_cliente, C.nombreEmpresa, P.fecha
  FROM Clientes C
        JOIN Pedidos P ON C.id_cliente = P.id_cliente
        JOIN Empleados E ON P.id_empleado = E.id_empleado
WHERE C.pais = 'Argentina' OR E.id_empleado = 5;
```

SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

CONSULTA DE DATOS – Consultas a más de una tabla

SQL permite la combinación externa (**OUTER JOIN**) para mantener los valores de la tabla que se puso a la izquierda, derecha o ambas, en los casos en los que no se cumplan las condiciones de vínculo entre tablas.

SELECT columnas_a_seleccionar

FROM T1 **[LEFT | RIGHT | FULL] [OUTER] JOIN** T2 **ON** condiciones_vínculos_tablas

[WHERE condiciones_de_búsqueda];

Ejemplo:

```
SELECT E.nombre_empleado, TE.telefono_número
   FROM empleados E LEFT OUTER JOIN telefonosEmp TE
                                ON E.id_empleado = TE.id_empleado;
```

LEFT (OUTER) JOIN: Para cada fila en T1 que no satisface la condición de vínculo con cualquier fila en T2, se agrega una fila unida con valores nulos en columnas de T2. El resultado tiene al menos una fila para cada fila en T1.

RIGHT (OUTER) JOIN: Para cada fila en T2 que no satisface la condición de vínculo con cualquier fila en T1, se agrega una fila unida con valores nulos en columnas de T1. El resultado tiene al menos una fila para cada fila en T2.

FULL (OUTER) JOIN: Agrega filas unidas con valores nulos para ambas tablas cuando no se satisfacen las condiciones de vínculo.